

SCREEPS OMEGA OPERATOR SERIES

Omega Operator Command Manual

Runtime Console Reference
Operations, Market, Power, Logistics, and Room Workflows

Repository: screeps_omega
Repository Version: 2751b35911042a164c0b0a3dd7438ad325f9daf8
Generated: 2026-06-17 23:16:00 PDT

PREPARED FOR PRINT REFERENCE

Omega Operator Command Manual

Repository: `screeps_omega`

Repository version used: 2751b35911042a164c0b0a3dd7438ad325f9daf8

Generation date: 2026-06-17 23:16:00 PDT

Source authority: `src/main.js`, `src/kernel_loop.js`, `src/ops.js`, `src/transfer_manager.js`, `src/market_console.js`, `src/room_reporting.js`, `src/pcl_manager.js`, `src/power_manager.js`, `src/empire_manager.js`, `src/reservation_manager.js`, `src/attack_manager.js`, `src/ops_logistics_manager.js`, `src/terminal_balance_manager.js`, and `src/config.js`.

This manual documents commands discovered from the source tree. Existing documentation was not used as command authority.

1. Getting Started

Omega exposes its operator surface through runtime globals registered every tick by `src/main.js:ops` and `market`. `src/ops.js` also registers global `view(mode)` plus convenience constants `on` and `off`.

`src/market_console.js` registers `market`.

Command philosophy:

- Use `ops` for empire, room, logistics, reservation, attack, power, Power Creep, observer, CPU, and reporting workflows.
- Use `market` for market visibility, terminal staging, dry-run planning, market execution preflight, explicit market execution, and history.
- Most commands print operator-readable console output and return either that printed string, a lines array, or a structured result object.
- Commands that mutate memory are still operator commands: they create plans, set policies, stage logistics requests, cancel requests, or update console workflow memory.

Approval-gated workflow:

- Power Creep actions use a `check` versus `confirm` pattern. `check` reports readiness and does not call the Screeps API action. `confirm` is required before commands call `powerCreep.usePower(...)` or `powerCreep.enableRoom(...)`.
- Market plan execution is a two-step workflow: create or inspect a saved plan, run `market.executionDryRun(planId)`, then intentionally run `market.executePlan(planId)`.
- Staged room-to-room transfers use `ops.transfer(..., "check")` versus `ops.transfer(..., "confirm")`. `check` reports readiness only. `confirm` creates a bounded transfer plan in `Memory.ops.transfers`; Omega advances approved plans each tick through room-local logistics requests and terminal sends.
- Direct manual market commands `market.buy(...)`, `market.sell(...)`, and `market.send(...)` execute immediately after local validation. They are not dry-run commands.

Execution levels used in this manual:

- Read Only: reads runtime or Memory state and prints a report.
- Planning: writes Memory, creates a plan, creates a logistics request, changes policy, or clears stored workflow/log history without directly calling market deal, terminal send, or Power Creep APIs.
- Approval Required: requires an explicit `confirm` or saved-plan execution command before calling an executing Screeps API.
- Executes Immediately: directly calls an execution API such as `Game.market.deal`, `terminal.send`, `powerCreep.usePower`, or `powerCreep.enableRoom`.

2. Global Command Index

Global aliases and constants

- `view(on|off)`

- `on`
- `off`

Ops commands

- `ops.attack(targetRoom, [postAction], [parentRoom], [allies])`
- `ops.attacks()`
- `ops.balanceTerminal(roomName)`
- `ops.balanceTerminals()`
- `ops.cancel(requestId)`
- `ops.cancelAttack(targetRoom)`
- `ops.cancelExpansion(targetRoom)`
- `ops.cancelRequests(roomName, "blocked", [filters])`
- `ops.cancelReserve(targetRoom)`
- `ops.clearTerminal(roomName, [resource], [amount])`
- `ops.cpu([roomName])`
- `ops.cpuStatus(roomName)`
- `ops.empire()`
- `ops.expand(targetRoom, [parentRoom])`
- `ops.expansions()`
- `ops.fillTerminal(roomName, resource, amount)`
- `ops.help()`
- `ops.hud(on|off)`
- `ops.log([roomName], [limit])`
- `ops.logClear([roomName])`
- `ops.move(resource, amount, roomName, from, to)`
- `ops.operator(name, [roomName], ["powers"])`
- `ops.operator(name, roomName, "operateExtension", mode)`
- `ops.operator(name, roomName, "operateSpawn", [spawnNameOrId], mode)`

- `ops.ops([roomName], ["stage", from, to, amount])`
- `ops.pcl([roomName])`
- `ops.phase(roomName)`
- `ops.power([roomName], [mode], [on|off])`
- `ops.powerCreep(name, "assign", room)`
- `ops.powerCreep(name, "generateOps", mode)`
- `ops.powerCreep(name, "renewAssist", on|off)`
- `ops.powerCreep(name, "renewStatus")`
- `ops.powerCreep(name, action, room, [target], [mode])`
- `ops.powerCreeps()`
- `ops.powerEnable(roomName, mode, [name])`
- `ops.reports(on|off)`
- `ops.reserve(targetRoom, [parentRoom])`
- `ops.reserved([parentRoom])`
- `ops.room([roomName], [section])`
- `ops.rooms()`
- `ops.requests([roomName], ["blocked"|"all"|"history"])`
- `ops.terminalStatus([roomName])`
- `ops.tickRate([sampleTicks|status|cancel])`
- `ops.tickSpeed([sampleTicks|status|cancel])`
- `ops.transfer(resource, amount, fromRoom, fromLocation, toRoom, toLocation, mode)`
- `ops.transfers()`
- `ops.transferStatus(id)`
- `ops.cancelTransfer(id)`

Market commands

- `market.buy(resource, amount, roomName)`
- `market.buyOptions([resource])`
- `market.cancel(requestId)`
- `market.clearExecutionLimit(name)`
- `market.clearHistory(mode)`
- `market.clearLimit(name)`
- `market.clearPlan(planId)`
- `market.clearPlan("all")`
- `market.clearPlans()`
- `market.deletePlan(planId)`
- `market.execute(planId)`
- `market.executePlan(planId)`
- `market.executionDryRun(planId)`
- `market.executionLimits()`
- `market.executionStatus()`
- `market.help()`
- `market.history([resource|roomName|"all"])`
- `market.historyAudit()`
- `market.historyLimit()`
- `market.historySummary()`
- `market.info()`
- `market.install()`
- `market.limits()`
- `market.needs()`
- `market.opportunities([resource])`
- `market.ping()`

- `market.plan(planId)`
- `market.planAudit()`
- `market.planBuy(resource, amount, roomName)`
- `market.planBuys()`
- `market.planReview(planId)`
- `market.planSell(resource, amount, roomName)`
- `market.planSells()`
- `market.planSummary()`
- `market.plans(["all"|"history"])`
- `market.readiness([roomName|resource])`
- `market.recommendations()`
- `market.removePlan(planId)`
- `market.requests([roomName], ["all"|"history"])`
- `market.restore()`
- `market.rooms()`
- `market.sell(resource, amount, roomName)`
- `market.sellOptions([resource])`
- `market.send(resource, amount, fromRoom, toRoom)`
- `market.setExecutionLimit(name, value)`
- `market.setHistoryLimit(limit)`
- `market.setLimit(name, value)`
- `market.stage(resource, amount, roomName)`
- `market.stock([roomName])`
- `market.surplus()`
- `market.uninstall()`
- `market.unstage(resource, amount, roomName)`

3. Command Reference

Global

Command	Description	Parameters	Example	Returns	Side Effects	Execution Level
<code>view(mode)</code>	Toggle HUD and critical reports together and optionally print the current room report when enabling.	<code>mode</code> : boolean, number, or string toggle.	<code>view(on)</code>	Object with enabled, HUD, and report state.	Updates ops view, HUD, and report flags.	Planning
<code>on</code>	Convenience global value for <code>true</code> .	None.	<code>ops.hud(on)</code>	Boolean constant.	None.	Read Only
<code>off</code>	Convenience global value for <code>false</code> .	None.	<code>ops.reports(off)</code>	Boolean constant.	None.	Read Only

Ops

Command	Description	Parameters	Example	Returns	Side Effects	Execution Level
<code>ops.help()</code>	Print the ops help surface.	None.	<code>ops.help()</code>	Help row array.	Prints help.	Read Only
<code>ops.hud(mode)</code>	Toggle the room HUD overlay.	<code>mode</code> : <code>on</code> , <code>off</code> , boolean, or equivalent.	<code>ops.hud(on)</code>	Toggle result object.	Updates HUD flag in ops state.	Planning
<code>ops.reports(mode)</code>	Toggle critical room reports.	<code>mode</code> : <code>on</code> , <code>off</code> , boolean, or equivalent.	<code>ops.reports(off)</code>	Toggle result object.	Updates reports flag in ops state.	Planning
<code>ops.room(arg1, arg2)</code>	Show one room report. If one argument is a section, uses the current/default room.	<code>arg1</code> : room name or section. <code>arg2</code> : section. Sections: overview, economy, build, defense, creeps, sources, advanced, power, observer, cpu, resources, all.	<code>ops.room("W5N5", "power")</code>	Report object, except CPU returns a printable string.	Sets current room and may update room progress.	Read Only
<code>ops.cpu(roomName)</code>	Show measured room CPU, top section costs, pressure, and scheduler skips.	<code>roomName</code> : optional owned room.	<code>ops.cpu("W5N5")</code>	Printable CPU report status string.	Sets current room through <code>ops.room</code> .	Read Only
<code>ops.cpuStatus(roomName)</code>	Alias for <code>ops.room(roomName, "cpu")</code> .	<code>roomName</code> : optional owned room.	<code>ops.cpuStatus("W5N5")</code>	Printable CPU report status string.	Sets current room through <code>ops.room</code> .	Read Only
<code>ops.phase(roomName)</code>	Alias for the build section of <code>ops.room</code> .	<code>roomName</code> : optional owned room.	<code>ops.phase("W5N5")</code>	Room report object for build section.	Sets current room through <code>ops.room</code> .	Read Only
<code>ops.rooms()</code>	Show overview lines for all owned rooms.	None.	<code>ops.rooms()</code>	Room reports array.	Updates progress in generated reports.	Read Only
<code>ops.empire()</code>	Show empire summary and owned-room overview.	None.	<code>ops.empire()</code>	Empire report object.	Updates progress in generated reports.	Read Only
<code>ops.log(arg1, arg2)</code>	Show compact invasion history.	<code>arg1</code> : room name or limit. <code>arg2</code> : optional limit.	<code>ops.log("W43N6")</code>	Lines array.	Prints invasion log lines.	Read Only
<code>ops.logClear(roomName)</code>	Clear invasion history for one room, or all rooms when omitted.	<code>roomName</code> : optional room or <code>all</code> .	<code>ops.logClear("W43N6")</code>	Clear result object.	Deletes invasion log entries.	Planning
<code>ops.tickRate(sampleTicks)</code>	Sample wall-clock milliseconds per tick over a short window; supports status and cancel.	<code>sampleTicks</code> : positive integer, <code>status</code> , or <code>cancel</code> .	<code>ops.tickRate(5)</code>	Printable status line.	Stores or clears probe state under runtime ops console memory.	Planning
<code>ops.tickSpeed(sampleTicks)</code>	Alias for <code>ops.tickRate</code> .	Same as <code>ops.tickRate</code> .	<code>ops.tickSpeed("status")</code>	Printable status line.	Same as <code>ops.tickRate</code> .	Planning
<code>ops.power(roomName, arg1, arg2)</code>	Show empire Power Spawn status, show a room power report, or set room-local processing/refill policy.	<code>roomName</code> : optional room. <code>arg1</code> : detail, <code>on</code> , <code>off</code> , <code>process</code> , <code>refill</code> , or <code>reserve</code> amount. <code>arg2</code> : toggle or amount depending on mode.	<code>ops.power("W5N5", "process", "off")</code>	Printable report or policy line.	Policy forms update room power policy memory.	Planning
<code>ops.pcl(roomName)</code>	Show GPL/PCL status and optional room enablement readiness.	<code>roomName</code> : optional room.	<code>ops.pcl("W5N5")</code>	Printable report string.	Prints report.	Read Only
<code>ops.powerCreeps()</code>	List friendly Power Creeps without controlling them.	None.	<code>ops.powerCreeps()</code>	Printable report string.	Prints report.	Read Only
<code>ops.operator(powerCreepName, roomName, mode, targetOrMode, maybeMode)</code>	Report operator readiness or check/confirm manual OPERATE SPAWN and OPERATE_EXTENSION.	<code>powerCreepName</code> : name. <code>roomName</code> : room. <code>mode</code> : <code>powers</code> , <code>operateSpawn</code> , or <code>operateExtension</code> . Additional target/mode args as needed.	<code>ops.operator("OperatorOne", "W5N5", "operateSpawn", "Spawn1", "check")</code>	Printable report string.	<code>confirm</code> for operate commands calls <code>powerCreep.usePower</code> ; report mode does not.	Approval Required
<code>ops.powerCreep(powerCreepName, action, roomName, targetOrMode, mode)</code>	Assign, unassign, report, check, or confirm Power Creep lifecycle and movement actions.	<code>action</code> : <code>assign</code> , <code>unassign</code> , <code>renewAssist</code> , <code>renewStatus</code> , <code>spawn</code> , <code>renew</code> , <code>position</code> , <code>move</code> , <code>generateOps</code> .	<code>ops.powerCreep("OperatorOne", "move", "W5N5", "powerSpawn", "check")</code>	Printable line or report string.	Assignment and renew-assist update memory. <code>confirm</code> lifecycle/generateOps calls Power Creep API actions.	Approval Required
<code>ops.ops(roomName, action, from, to, amount, powerCreepName)</code>	Show ops resource inventory or stage manual RESOURCE_OPS between room storage and terminal.	<code>roomName</code> : optional room. <code>action</code> : optional stage. <code>from/to</code> : storage or terminal. <code>amount</code> : positive number.	<code>ops.ops("W5N5", "stage", "storage", "terminal", 1000)</code>	Printable inventory block or request line.	Stage creates an ops logistics move request.	Planning
<code>ops.powerEnable(roomName, mode, powerCreepName)</code>	Check room power enablement readiness or confirm <code>enableRoom</code> .	<code>roomName</code> : owned room. <code>mode</code> : <code>check</code> or <code>confirm</code> . <code>powerCreepName</code> : required when confirming if more than one candidate exists.	<code>ops.powerEnable("W5N5", "check")</code>	Printable report string.	<code>confirm</code> calls <code>powerCreep.enableRoom</code> .	Approval Required
<code>ops.move(resource, amount, roomName, from, to)</code>	Create a room-local logistics request between storage and terminal.	<code>resource</code> : resource constant or string. <code>amount</code> : positive number. <code>roomName</code> : room. <code>from/to</code> : endpoints.	<code>ops.move("H", 50000, "W42N9", "terminal", "storage")</code>	Logistics result object.	Creates a move request in ops logistics memory.	Planning
<code>ops.transfer(resource, amount, fromRoom, fromLocation, toRoom, toLocation, mode)</code>	Check or confirm an explicit staged transfer plan. Supports storage -> terminal, terminal -> storage, storage -> storage, and terminal -> terminal.	<code>resource</code> : resource constant or string. <code>amount</code> : positive number. <code>fromRoom/toRoom</code> : room names. <code>fromLocation/toLocation</code> : storage or terminal. <code>mode</code> : <code>check</code> or <code>confirm</code> ; omitted mode defaults to <code>check</code> .	<code>ops.transfer(RESOURCE_POWER, 1000, "W41N7", "storage", "W42N9", "storage", "confirm")</code>	Printable transfer line.	<code>check</code> is report-only. <code>confirm</code> creates a plan in <code>Memory.ops.transfers</code> ; approved plans may create ops logistics requests and call <code>terminal.send</code> as they advance each tick.	Approval Required
<code>ops.transfers()</code>	Show active staged transfer plans.	None.	<code>ops.transfers()</code>	Printable summary block.	Prints active transfer summaries.	Read Only
<code>ops.transferStatus(id)</code>	Show detailed status for one staged transfer plan.	<code>id</code> : transfer plan id.	<code>ops.transferStatus("ot_123_W41N7_W42N9_4567")</code>	Printable detail block.	Prints plan detail and may refresh delivered/completion progress from visible storage.	Read Only
<code>ops.cancelTransfer(id)</code>	Cancel an active staged transfer plan.	<code>id</code> : transfer plan id.	<code>ops.cancelTransfer("ot_123_W41N7_W42N9_4567")</code>	Printable cancellation line.	Marks the transfer CANCELLED and cancels associated source/destination ops logistics requests when present.	Planning
<code>ops.terminalStatus(roomName)</code>	Show terminal capacity, energy, resources, and congestion status.	<code>roomName</code> : optional owned room.	<code>ops.terminalStatus("W42N9")</code>	Printable block.	Prints report.	Read Only
<code>ops.clearTerminal(roomName, resource, amount)</code>	Create terminal-to-storage logistics requests for terminal cleanup.	<code>roomName</code> : owned room. Optional <code>resource</code> and <code>amount</code> ; when omitted, source selects cleanup resources.	<code>ops.clearTerminal("W42N9", "H", 50000)</code>	Result object or cleanup summary.	Creates one or more logistics requests.	Planning
<code>ops.fillTerminal(roomName, resource, amount)</code>	Create a storage-to-terminal logistics request for market staging.	<code>roomName</code> , <code>resource</code> , <code>amount</code> .	<code>ops.fillTerminal("W42N9", "energy", 10000)</code>	Logistics result object.	Creates a logistics request.	Planning
<code>ops.requests(roomName, mode)</code>	Show active, blocked-only, or historical ops logistics requests.	<code>roomName</code> : optional room. <code>mode</code> : optional blocked, all, or history; blocked can also be the first argument.	<code>ops.requests("W42N9", "blocked")</code>	Printable block with counts, age, reason, created, and updated fields.	Prints report.	Read Only
<code>ops.cancelRequests(roomName, status, filters)</code>	Cancel stale blocked unclaimed ops logistics requests for one room.	<code>roomName</code> : owned room. <code>status</code> : only blocked. <code>filters</code> : optional resource or resourceType, <code>from</code> , <code>to</code> , and <code>olderThan</code> ; default age is 1000 ticks.	<code>ops.cancelRequests("W42N9", "blocked", { resource: RESOURCE_POWER, from: "terminal", to: "powerSpawn", olderThan: 1000 })</code>	Printable cleanup block.	Marks matching blocked, unclaimed requests as canceled; skips open, done, expired, canceled, claimed, or non-matching requests.	Planning
<code>ops.cancel(requestId)</code>	Cancel an ops logistics request.	<code>requestId</code> : request id.	<code>ops.cancel("ol_123_W42N9_H_1")</code>	Cancel result object.	Marks request canceled.	Planning
<code>ops.balanceTerminal(roomName)</code>	Evaluate one terminal balance target set and create conservative logistics requests.	<code>roomName</code> : owned room.	<code>ops.balanceTerminal("W42N9")</code>	Terminal balance result object.	May create logistics requests.	Planning
<code>ops.balanceTerminals()</code>	Evaluate terminal balance targets for all owned rooms with storage and terminal.	None.	<code>ops.balanceTerminals()</code>	Aggregate result object.	May create logistics requests.	Planning
<code>ops.expand(targetRoom, parentRoom)</code>	Start or update a manual expansion plan.	<code>targetRoom</code> : room to claim. <code>parentRoom</code> : optional support room.	<code>ops.expand("W5N6", "W5N5")</code>	Expansion result object.	Creates/updates expansion memory; may convert reservation to expansion.	Planning

Command	Description	Parameters	Example	Returns	Side Effects	Execution Level
<code>ops.reserve(targetRoom, parentRoom)</code>	Start or update a reserved-room plan.	<code>targetRoom</code> : remote room. <code>parentRoom</code> : optional parent; current room is used when omitted.	<code>ops.reserve("W5N6", "W5N5")</code>	Reservation result object.	Creates/updates reservation memory; may convert expansion to reservation.	Planning
<code>ops.reserved(parentRoom)</code>	Show active reserved rooms grouped by parent.	<code>parentRoom</code> : optional parent filter.	<code>ops.reserved("W5N5")</code>	Lines array.	Prints report.	Read Only
<code>ops.expansions()</code>	Show active expansion plans.	None.	<code>ops.expansions()</code>	Lines array.	Prints report.	Read Only
<code>ops.attack(targetRoom, postActionOrOptions, parentRoom, allies)</code>	Start or update a manual attack plan.	<code>targetRoom</code> : target. <code>postAction</code> : expand, reserve, or none; options object also accepted. <code>parentRoom</code> : support room. <code>allies</code> : array.	<code>ops.attack("W5N6", "expand", "W5N5", ["W4N6"])</code>	Attack result object.	Creates/updates attack memory.	Planning
<code>ops.attacks()</code>	Show active attack plans.	None.	<code>ops.attacks()</code>	Lines array.	Prints report.	Read Only
<code>ops.cancelAttack(targetRoom)</code>	Cancel an active attack plan.	<code>targetRoom</code> : target room.	<code>ops.cancelAttack("W5N6")</code>	Cancel result object.	Cancels attack plan memory.	Planning
<code>ops.cancelExpansion(targetRoom)</code>	Cancel an active expansion plan.	<code>targetRoom</code> : target room.	<code>ops.cancelExpansion("W5N6")</code>	Cancel result object.	Cancels expansion plan memory.	Planning
<code>ops.cancelReserve(targetRoom)</code>	Cancel an active reserved-room plan.	<code>targetRoom</code> : reserved room.	<code>ops.cancelReserve("W5N6")</code>	Cancel result object.	Cancels reservation plan memory.	Planning

Market

Command	Description	Parameters	Example	Returns	Side Effects	Execution Level
<code>market.help()</code>	Print the market command help surface.	None.	<code>market.help()</code>	Printable block.	Prints help.	Read Only
<code>market.info()</code>	Show helper runtime and memory status.	None.	<code>market.info()</code>	Object with runtime status and memory.	Touches market console memory registration fields.	Planning
<code>market.ping()</code>	Confirm market helper availability.	None.	<code>market.ping()</code>	Printable pong line.	Prints line.	Read Only
<code>market.install()</code>	Install and register market helper in runtime memory.	None.	<code>market.install()</code>	Printable line.	Updates market memory and registers global.	Planning
<code>market.restore()</code>	Restore a previously uninstalled helper.	None.	<code>market.restore()</code>	Printable line.	Updates market memory and registers global.	Planning
<code>market.uninstall()</code>	Mark helper uninstalled and delete <code>global.market</code> for current runtime.	None.	<code>market.uninstall()</code>	Printable line.	Updates memory and deletes runtime global.	Planning
<code>market.rooms()</code>	List owned rooms and terminal availability.	None.	<code>market.rooms()</code>	Printable block.	Prints report.	Read Only
<code>market.stock(roomName)</code>	Show empire or room storage/terminal stock.	<code>roomName</code> : optional owned room.	<code>market.stock("W42N9")</code>	Printable block.	Prints report.	Read Only
<code>market.needs()</code>	Show obvious room-level market/logistics needs.	None.	<code>market.needs()</code>	Printable block.	Prints report.	Read Only
<code>market.surplus()</code>	Show obvious surplus resources.	None.	<code>market.surplus()</code>	Printable block.	Prints report.	Read Only
<code>market.stage(resource, amount, roomName)</code>	Stage a resource from storage to terminal.	<code>resource</code> , <code>amount</code> , <code>roomName</code> .	<code>market.stage("H", 10000, "W42N9")</code>	Printable logistics message.	Creates ops logistics request.	Planning
<code>market.unstage(resource, amount, roomName)</code>	Move a resource from terminal to storage.	<code>resource</code> , <code>amount</code> , <code>roomName</code> .	<code>market.unstage("H", 10000, "W42N9")</code>	Printable logistics message.	Creates ops logistics request.	Planning
<code>market.requests(roomName, mode)</code>	Show market-compatible logistics requests.	<code>roomName</code> : optional. <code>mode</code> : optional <code>all</code> or <code>history</code> .	<code>market.requests("W42N9", "all")</code>	Printable block.	Prints report.	Read Only
<code>market.cancel(requestId)</code>	Cancel an ops or market request by id.	<code>requestId</code> : request id.	<code>market.cancel("ol_123_W42N9_H_1")</code>	Printable cancel message.	Marks request canceled when found.	Planning
<code>market.send(resource, amount, fromRoom, toRoom)</code>	Send resources directly between owned terminals.	<code>resource</code> , <code>amount</code> , <code>fromRoom</code> , <code>toRoom</code> .	<code>market.send("energy", 5000, "W42N9", "W41N8")</code>	Printable send result.	Calls <code>terminal.send</code> .	Executes Immediately
<code>market.buyOptions(resource)</code>	Show usable buy options from sell orders.	<code>resource</code> : optional resource.	<code>market.buyOptions("energy")</code>	Printable block.	Reads market orders.	Read Only
<code>market.sellOptions(resource)</code>	Show usable sell options against buy orders.	<code>resource</code> : optional resource.	<code>market.sellOptions("H")</code>	Printable block.	Reads market orders.	Read Only
<code>market.readiness(input)</code>	Show market readiness by empire, room, or resource.	<code>input</code> : optional room name or resource.	<code>market.readiness("H")</code>	Printable block.	Prints report.	Read Only
<code>market.opportunities(resource)</code>	Show ready market sell opportunities.	<code>resource</code> : optional resource.	<code>market.opportunities("H")</code>	Printable block.	Reads market and terminal state.	Read Only
<code>market.recommendations()</code>	Show conservative next manual market/logistics commands.	None.	<code>market.recommendations()</code>	Printable block.	Prints report.	Read Only
<code>market.planSell(resource, amount, roomName)</code>	Save a dry-run sell plan.	<code>resource</code> , <code>amount</code> , <code>roomName</code> .	<code>market.planSell("H", 10000, "W42N9")</code>	Printable plan report.	Writes saved plan memory.	Planning
<code>market.planBuy(resource, amount, roomName)</code>	Save a dry-run buy plan.	<code>resource</code> , <code>amount</code> , <code>roomName</code> .	<code>market.planBuy("energy", 5000, "W41N8")</code>	Printable plan report.	Writes saved plan memory.	Planning
<code>market.plans(mode)</code>	List saved plans.	<code>mode</code> : optional <code>all</code> or <code>history</code> .	<code>market.plans("all")</code>	Printable block.	May mark expired plans during listing.	Planning
<code>market.plan(planId)</code>	Recheck and show one saved plan.	<code>planId</code> : saved plan id.	<code>market.plan("ms_223455_H_W42N9")</code>	Printable plan report.	May update plan status.	Planning
<code>market.planSummary()</code>	Summarize saved plan counts.	None.	<code>market.planSummary()</code>	Printable block.	Reads plan memory.	Read Only
<code>market.planReview(planId)</code>	Review saved plan readiness before execution.	<code>planId</code> : saved plan id.	<code>market.planReview("ms_223455_H_W42N9")</code>	Printable review report.	May update plan status.	Planning
<code>market.planAudit()</code>	Audit active saved plans for stale, duplicate, blocked, or missing-order cases.	None.	<code>market.planAudit()</code>	Printable block.	May update stale status through rechecks.	Planning
<code>market.clearPlan(planId)</code>	Soft-delete one plan or active plans.	<code>planId</code> or <code>all</code> .	<code>market.clearPlan("all")</code>	Printable line.	Marks plan(s) deleted.	Planning
<code>market.deletePlan(planId)</code>	Deprecated alias for <code>market.clearPlan(planId)</code> .	<code>planId</code> : saved plan id.	<code>market.deletePlan("ms_223455_H_W42N9")</code>	Printable deprecation line and clear result.	Marks plan deleted.	Planning
<code>market.removePlan(planId)</code>	Deprecated alias for <code>market.clearPlan(planId)</code> .	<code>planId</code> : saved plan id.	<code>market.removePlan("ms_223455_H_W42N9")</code>	Printable deprecation line and clear result.	Marks plan deleted.	Planning
<code>market.clearPlans()</code>	Deprecated alias for <code>market.clearPlan("all")</code> .	None.	<code>market.clearPlans()</code>	Printable deprecation line and clear result.	Marks active plans deleted.	Planning
<code>market.executionStatus()</code>	Show execution design and safety status.	None.	<code>market.executionStatus()</code>	Printable block.	Prints report.	Read Only
<code>market.executionDryRun(planId)</code>	Recompute execution preflight without executing.	<code>planId</code> : saved plan id.	<code>market.executionDryRun("ms_223455_H_W42N9")</code>	Printable preflight report.	Reads current market/terminal state.	Read Only
<code>market.executionLimits()</code>	Show configured execution limits.	None.	<code>market.executionLimits()</code>	Printable block.	Ensures default limit memory exists.	Planning
<code>market.setExecutionLimit(name, value)</code>	Set a numeric execution safety limit.	<code>name</code> : supported limit. <code>value</code> : number or null for <code>maxBuyEffectivePrice</code> .	<code>market.setExecutionLimit("maxSellAmount", 5000)</code>	Printable line.	Updates execution limit memory.	Planning
<code>market.clearExecutionLimit(name)</code>	Reset one execution limit to its default.	<code>name</code> : supported limit.	<code>market.clearExecutionLimit("maxSellAmount")</code>	Printable line.	Updates execution limit memory.	Planning
<code>market.limits()</code>	Alias for <code>market.executionLimits()</code> .	None.	<code>market.limits()</code>	Printable block.	Ensures default limit memory exists.	Planning
<code>market.setLimit(name, value)</code>	Alias for <code>market.setExecutionLimit</code> .	<code>name</code> , <code>value</code> .	<code>market.setLimit("maxBuyAmount", 5000)</code>	Printable line.	Updates execution limit memory.	Planning
<code>market.clearLimit(name)</code>	Alias for <code>market.clearExecutionLimit</code> .	<code>name</code> .	<code>market.clearLimit("maxBuyAmount")</code>	Printable line.	Updates execution limit memory.	Planning

Command	Description	Parameters	Example	Returns	Side Effects	Executi on Level
<code>market.executePlan(planId)</code>	Execute a saved market plan after current-state preflight passes.	<code>planId</code> : saved plan id.	<code>market.executePlan("ms_223455_H_W42N9")</code>	Printable execution report.	Calls <code>Game.market.deal</code> ; appends history and updates plan execution fields.	Approval Require d
<code>market.execute(planId)</code>	Alias for <code>market.executePlan(planId)</code> .	<code>planId</code> : saved plan id.	<code>market.execute("ms_223455_H_W42N9")</code>	Printable execution report.	Calls <code>Game.market.deal</code> ; appends history and updates plan execution fields.	Approval Require d
<code>market.history(filter)</code>	Show recent market execution history.	<code>filter</code> : optional resource, room, or <code>all</code> .	<code>market.history("H")</code>	Printable block.	Trims history to current limit.	Plannin g
<code>market.historySummary()</code>	Summarize market execution history counts and accounting.	None.	<code>market.historySummary()</code>	Printable block.	Reads history.	Read Only
<code>market.historyAudit()</code>	Audit history for repeated failures, stale plans, and limit blocks.	None.	<code>market.historyAudit()</code>	Printable block.	Reads history.	Read Only
<code>market.clearHistory(mode)</code>	Remove selected execution history entries.	<code>mode</code> : <code>failed</code> , <code>blocked</code> , <code>stale</code> , <code>deleted</code> , or <code>all</code> .	<code>market.clearHistory("failed")</code>	Printable line.	Deletes history entries.	Plannin g
<code>market.setHistoryLimit(limit)</code>	Set maximum market history size.	<code>limit</code> : integer at least 10.	<code>market.setHistoryLimit(100)</code>	Printable line.	Updates history limit and trims old entries.	Plannin g
<code>market.historyLimit()</code>	Show current history retention limit.	None.	<code>market.historyLimit()</code>	Printable line.	Ensures default history limit exists.	Plannin g
<code>market.buy(resource, amount, roomName)</code>	Directly buy a resource from the best usable sell order.	<code>resource</code> , <code>amount</code> , <code>roomName</code> .	<code>market.buy("energy", 5000, "W41N8")</code>	Printable execution line.	Calls <code>Game.market.deal</code> .	Executes Immedi ately
<code>market.sell(resource, amount, roomName)</code>	Directly sell a resource to the best usable buy order.	<code>resource</code> , <code>amount</code> , <code>roomName</code> .	<code>market.sell("H", 2580, "W42N9")</code>	Printable execution line.	Calls <code>Game.market.deal</code> .	Executes Immedi ately
<code>market.planBuys()</code>	Print suggested buy or receive actions.	None.	<code>market.planBuys()</code>	Printable block.	Prints report.	Read Only
<code>market.planSells()</code>	Print suggested sell or send actions.	None.	<code>market.planSells()</code>	Printable block.	Prints report.	Read Only

4. Room Operations

Economy:

- `ops.room({roomName}, "economy")`: room energy, storage, hub/controller container, upgrade rate, spawn queue, and hauler mode.
- `ops.room({roomName}, "resources")`: storage energy, terminal energy, terminal power, ghodium, minerals, and terminal balance state.
- `ops.terminalStatus({roomName})`: terminal stock and congestion status.
- `market.stock({roomName})`, `market.needs()`, and `market.surplus()`: economy visibility from the market helper.

Logistics:

- `ops.move(resource, amount, roomName, from, to)`: create storage/terminal movement requests.
- `ops.transfer(resource, amount, fromRoom, fromLocation, toRoom, toLocation, "check")`: preview an explicit staged cross-room transfer without creating a plan.
- `ops.transfer(resource, amount, fromRoom, fromLocation, toRoom, toLocation, "confirm")`: approve what should move; Omega advances the plan each tick through source staging, terminal send, destination staging, completion, block, or cancellation states.
- `ops.transfers()`, `ops.transferStatus(id)`, and `ops.cancelTransfer(id)`: list, inspect, and cancel staged transfers. The command is `ops.cancelTransfer(id)`; `ops.transferCancel(id)` is not implemented.
- `ops.clearTerminal(...)`, `ops.fillTerminal(...)`, `ops.balanceTerminal(...)`, and `ops.balanceTerminals()`: terminal hygiene and balance workflows.
- `ops.requests(...)`, `ops.requests("blocked")`, `ops.cancelRequests(room, "blocked", filters)`, `ops.cancel(...)`, `market.stage(...)`, `market.unstage(...)`, `market.requests(...)`, and `market.cancel(...)`: logistics request visibility and control.

Construction:

- `ops.room({roomName}, "build")`: roadmap status, current/future phase work, storage planning, mineral road needs, and next task.
- `ops.phase(roomName)`: undocumented alias for the build section.

Remotes:

- `ops.reserve(targetRoom, [parentRoom])`, `ops.reserved([parentRoom])`, and `ops.cancelReserve(targetRoom)`: reserved-room lifecycle.
- `ops.expand(targetRoom, [parentRoom])`, `ops.expansions()`, and `ops.cancelExpansion(targetRoom)`: expansion lifecycle.
- `ops.attack(targetRoom, [postAction], [parentRoom], [allies])`, `ops.attacks()`, and `ops.cancelAttack(targetRoom)`: attack plan lifecycle and post-action planning.

Defense:

- `ops.room({roomName}, "defense")`: alert state, safe mode, hostiles, threat score, response mode, breach severity, tower status, defender readiness, support, and target focus.
- `ops.log({roomName}, [limit])` and `ops.logClear({roomName})`: invasion log inspection and cleanup.

CPU:

- `ops.cpu({roomName})` and `ops.cpuStatus({roomName})`: room CPU report.
- `ops.tickRate([sampleTicks|status|cancel])` and `ops.tickSpeed(...)`: wall-clock tick-speed probe.

Reporting:

- `ops.room({roomName}, "overview"|"all")`, `ops.rooms()`, and `ops.empire()`: primary room and empire reports.
- `ops.hud(on|off)`, `ops.reports(on|off)`, and `view(on|off)`: reporting visibility controls.

5. Power Operations

GPL:

- `ops.pcl({roomName})` prints global GPL/PCL status and optional room enablement readiness.

PCL:

- `ops.pcl({roomName})` is the PCL summary entry point.
- `ops.powerCreeps()` lists friendly Power Creeps.

Power Spawn:

- `ops.power()` shows empire Power Spawn status.
- `ops.power("ROOM", "detail")` or `ops.room("ROOM", "power")` shows a room power report.
- `ops.power("ROOM", "process", "on"|"off")`, `ops.power("ROOM", "refill", "on"|"off")`, and reserve forms set room-local Power Spawn policy.
- Power Spawn refill policy is owned by `power_manager`; execution uses `ops.logistics_manager` requests, and haulers execute those requests through the normal ops logistics priority. `advanced_structure_manager` reports Power Spawn readiness only and does not create Power Spawn refill haul tasks.

Power Creeps:

- `ops.powerCreep(name, "assign", room)` and `ops.powerCreep(name, "unassign")` manage renewal-assist assignment memory.
- `ops.powerCreep(name, "renewAssist", on|off)` toggles per-creep renewal assist.
- `ops.powerCreep(name, "renewStatus")` reports renewal assist state.
- `ops.powerCreep(name, "position", room)` reports target positioning.
- `ops.powerCreep(name, "move", room, target, "check"|"confirm")` checks or confirms movement preparation.
- `ops.powerCreep(name, "spawn", room, "check"|"confirm")`, `ops.powerCreep(name, "renew", room, "check"|"confirm")`, and `ops.powerCreep(name, "generateOps", "check"|"confirm")` are approval-gated lifecycle/power actions.

Operators:

- `ops.operator(name, [room], "powers")` reports `OPERATE_*` readiness and target visibility.
- `ops.operator(name, room, "operateSpawn", [spawnNameOrId], "check"|"confirm")` checks or confirms `PWR_OPERATE_SPAWN`.
- `ops.operator(name, room, "operateExtension", "check"|"confirm")` checks or confirms `PWR_OPERATE_EXTENSION`.

Ops resource management:

- `ops.ops()` shows empire ops inventory.
- `ops.ops("ROOM")` shows room ops inventory.
- `ops.ops("ROOM", "stage", "storage", "terminal", amount)` stages ops between storage and terminal through logistics requests.

6. Market Operations

Visibility:

- `market.rooms()`, `market.stock([roomName])`, `market.needs()`, and `market.surplus()`.

Internal terminal logistics:

- `market.stage(resource, amount, roomName)`, `market.unstage(resource, amount, roomName)`, `market.requests(...)`, `market.cancel(requestId)`, and `market.send(...)`.

Market scanning:

- `market.buyOptions([resource])` and `market.sellOptions([resource])`.

Market intelligence:

- `market.readiness([roomName|resource])`, `market.opportunities([resource])`, and `market.recommendations()`.

Dry-run planning:

- `market.planSell(...)`, `market.planBuy(...)`, `market.plans(...)`, `market.plan(...)`, `market.planSummary()`, `market.planReview(...)`, `market.planAudit()`, and `market.clearPlan(...)`.
- Deprecated aliases remain implemented: `market.deletePlan(...)`, `market.removePlan(...)`, and `market.clearPlans()`.

Execution design and safety:

- `market.executionStatus()`, `market.executionDryRun(planId)`, `market.executionLimits()`, `market.setExecutionLimit(name, value)`, `market.clearExecutionLimit(name)`, `market.limits()`, `market.setLimit(name, value)`, `market.clearLimit(name)`, `market.executePlan(planId)`, and `market.execute(planId)`.

Execution history:

- `market.history([filter])`, `market.historySummary()`, `market.historyAudit()`, `market.clearHistory(mode)`, `market.setHistoryLimit(limit)`, and `market.historyLimit()`.

Manual trading:

- `market.buy(resource, amount, roomName)` and `market.sell(resource, amount, roomName)` execute immediately through `Game.market.deal`.

7. Observer Operations

No `observer.*` command root is registered in source.

Observer reporting is exposed through room reports:

- `ops.room("observer")`: prints observer report for the current/default room.
- `ops.room("ROOM", "observer")`: prints observer report for a specific room.
- `ops.room("ROOM", "all")`: includes observer lines with the rest of the room report.

The observer section reports whether observer logic is enabled, visible observer count, last target, last result, queued targets, target list, intel count, newest/oldest intel age, run interval, and last run tick. Source: `src/room_reporting.js:1576`.

8. Troubleshooting

Cannot create first Power Creep:

- Run `ops.pcl("ROOM")` to inspect GPL/PCL and room enablement readiness.
- Run `ops.powerEnable("ROOM", "check")` before attempting confirmation.
- If readiness is blocked, inspect the printed next step from the enablement report.

Power Spawn not processing:

- Run `ops.power()` for empire summary.
- Run `ops.power("ROOM", "detail")` or `ops.room("ROOM", "power")`.
- Check global and room-local process/refill overrides, energy/power levels, terminal balance, refill pending count, and blocked reason.
- For refill stalls, inspect the `Refill owner power_manager | execution ops_logistics` line plus pending refill ids; stale blocked requests should be canceled or allowed to expire through existing ops logistics controls.
- Use `ops.power("ROOM", "process", "on")` or `ops.power("ROOM", "refill", "on")` only when you intend to change policy.

Remote reservation lost:

- Run `ops.reserved("PARENT")` to check active plans.
- Run `ops.room("PARENT", "defense")` and `ops.log("TARGET")` for threat context.
- Recreate or update with `ops.reserve("TARGET", "PARENT")`.

Room CPU high:

- Run `ops.cpu("ROOM")`.
- Review pressure, trend, scheduler skips, and top section cost lines.
- Run `ops.tickRate("status")` if a wall-clock probe is already active, or `ops.tickRate(5)` to sample.

Terminal clogged or market blocked:

- Run `ops.terminalStatus("ROOM")` and `market.readiness("ROOM")`.
- Use `ops.clearTerminal("ROOM")` for source-selected cleanup or `ops.clearTerminal("ROOM", resource, amount)` for a specific resource.
- Run `ops.requests("ROOM", "blocked")` or `market.requests("ROOM", "all")` to find stuck logistics.
- Use `ops.cancelRequests("ROOM", "blocked", filters)` only for stale blocked requests that are unclaimed and safe to clear.

Market plan will not execute:

- Run `market.planReview(planId)` and `market.executionDryRun(planId)`.
- Check stale order, terminal cooldown, resource amount, terminal energy, credits, and execution limits.
- Run `market.executionLimits()` before changing limits.

Power Creep operation does not fire:

- Run `ops.operator(name, room, "powers")`.
- Use `check` mode first for the specific action.
- Use `confirm` only when the check report is ready and the target is correct.

9. Quick Reference

Read-only first:

- Rooms: `ops.room("ROOM")`, `ops.room("ROOM", "economy")`, `ops.room("ROOM", "defense")`, `ops.rooms()`, `ops.empire()`
- CPU: `ops.cpu("ROOM")`, `ops.tickRate(5)`, `ops.tickRate("status")`
- Power: `ops.power()`, `ops.power("ROOM", "detail")`, `ops.pcl("ROOM")`, `ops.powerCreeps()`
- Observer: `ops.room("ROOM", "observer")`
- Market: `market.readiness()`, `market.opportunities("H")`, `market.recommendations()`

Planning and memory updates:

- Reservations: `ops.reserve("TARGET", "PARENT")`, `ops.cancelReserve("TARGET")`
- Expansions: `ops.expand("TARGET", "PARENT")`, `ops.cancelExpansion("TARGET")`
- Attack plans: `ops.attack("TARGET", "expand", "PARENT", ["ALLY"])`, `ops.cancelAttack("TARGET")`
- Logistics: `ops.move("H", 50000, "ROOM", "terminal", "storage")`, `ops.transfer(RESOURCE_POWER, 1000, "W41N7", "storage", "W42N9", "storage", "confirm")`, `ops.cancelRequests("ROOM", "blocked", { olderThan: 1000 })`, `ops.fillTerminal("ROOM", "energy", 10000)`, `ops.clearTerminal("ROOM")`
- Market plans: `market.planSell("H", 10000, "ROOM")`, `market.planReview(planId)`, `market.executionDryRun(planId)`

Executing commands:

- Power: `ops.powerCreep(name, "generateOps", "confirm")`, `ops.operator(name, "ROOM", "operateSpawn", "Spawn1", "confirm")`, `ops.powerEnable("ROOM", "confirm", name)`
- Market: `market.executePlan(planId)`, `market.buy(resource, amount, roomName)`, `market.sell(resource, amount, roomName)`, `market.send(resource, amount, fromRoom, toRoom)`

Command Surface Findings

Counts by category:

- Global callable aliases: 1 (view).
- Global convenience constants: 2 (on, off).
- `ops.*` callable functions: 42.
- `market.*` callable functions: 51.

- Total callable operator commands documented: 94.
- Ops help signature rows: 46.
- Market help signature rows: 65.

Missing documentation:

- `ops.tickSpeed(sampleTicks)` is implemented as an alias for `ops.tickRate` but has no ops help entry.
- `ops.cpuStatus(roomName)` is implemented as an alias for `ops.room(roomName, "cpu")` but has no ops help entry.
- `ops.phase(roomName)` is implemented as an alias for `ops.room(roomName, "build")` but has no ops help entry.
- `market.version` and `market.config` are exposed fields, not commands; they are intentionally excluded from the command count.

Duplicate commands:

- `ops.tickSpeed` duplicates `ops.tickRate`.
- `ops.cpuStatus` duplicates `ops.cpu`.
- `ops.phase` duplicates `ops.room(..., "build")`.
- `market.limits`, `market.setLimit`, and `market.clearLimit` duplicate the execution-limit commands.
- `market.execute` duplicates `market.executePlan`.
- `market.deletePlan`, `market.removePlan`, and `market.clearPlans` are deprecated aliases for `market.clearPlan`.

Stale help entries:

- No `ops.help()` or `market.help()` entry was found that references a missing callable implementation.
- Some help rows intentionally document variants of the same function, for example `market.stock()` and `market.stock(roomName)`.

Broken references:

- No broken help-to-implementation references were found in the source-registered help surfaces.
- No `attack.*`, `expansion.*`, `reservation.*`, `logistics.*`, `power.*`, `powerCreep.*`, `operator.*`, or `observer.*` global roots are registered. These workflows are exposed under `ops.*`.

Opportunities to consolidate command surfaces:

- Add help rows for implemented aliases or intentionally hide them as internal compatibility aliases.
- Consider documenting `ops.room` sections directly in `ops.help()` so `observer`, `resources`, and `advanced` are discoverable from the console.
- Consider aligning deprecated market plan aliases around one compatibility section so operators do not confuse aliases with preferred commands.
- Consider separating immediate market execution commands from approval-gated saved-plan execution in `market.help()` to make risk levels clearer.